Architecture Selection Justification

Lab 3 – Component Modelling & Architectural Pattern Selection

We chose Microservices Architecture for the Student Project Portfolio & Showcase App.

Architectural Choice: Layered and Client-Server were also evaluated. A layered monolith is simple but can only be scaled as a whole, and a single central server becomes a bottleneck and single point of failure under expo traffic. Microservices (Profile, OTP Verification, Voting, Judging, Leaderboard and Review Monitor services behind an API Gateway, connected by an Event Bus) match the scenario best.

Two Reasons

1. Independent scaling for peak load (NFR-001, NFR-002). Expo traffic is concentrated on voting and the leaderboard, with 3,000 concurrent users expected. These two services can be given extra instances while Profile and Judging stay small, instead of scaling the entire application.
2. Fault isolation and decoupled features (FR-003, FR-005). If the SMS/Email provider or GitHub API is slow, only OTP verification or profile creation is affected; browsing, judging and the leaderboard keep working. Fraud detection (Review Monitor) listens to events, so it never slows down a vote.

Security Advantage

The API Gateway is the single public entry point that handles JWT authentication, role-based access (Judge / Team / Voter) and rate limiting, which helps block scripted vote inflation (anti-sybil, FR-003). The OTP Verification Service is internal and reachable only from the Voting Service, and each service owns its own datastore, so a breach of one service does not expose votes, scores or OTP codes held by the others.

Performance Benefit

Voting is event-driven: the Voting Service records the vote, replies to the voter immediately and publishes a VoteCast event. The Leaderboard Service recalculates asynchronously and serves ranks from an in-memory cache, so read and write APIs can stay under the 300 ms target (NFR-001) even while ranks are being updated.

Trade-off Accepted

Microservices add operational complexity, network latency between services and eventual consistency of the leaderboard. For this system a rank that lags by a fraction of a second is acceptable, and the scaling and isolation benefits outweigh the extra complexity.